

Table: /health

Relative URL	Method	Required Body	Data section	Description
/health	GET	None	message: string	Health check endpoint to verify server is running
/monitoring	GET	None	running: bool, active_devices: number, total_entries: number, database_accessible: bool, last_update: string	Checks nlbwmon monitoring status on the router

Table: /api/session

Relative URL	Method	Required Body	Data section	Description
/start	POST	{ "sessionId": "<session_id>", "routerId": "<router_id>", "restart": false} }	session_id: string, router_reachable: bool, monitoring_ready: bool, message: string	Starts a new session for a router in headless mode (monitoring ensured)
/end	POST	{ "sessionId": "<session_id>", "routerId": "<router_id>" }	message: string	Ends the session for a router and cleans up connections
/refresh	POST	{ "sessionId": "<session_id>", "routerId": "<router_id>" }	message: string	Refreshes a session's activity timer
/status	GET	None	message: string, sessions: object	Retrieves the status of all active sessions

Table: /api/network

Relative URL	Method	Required Body	Data section	Description
/scan	GET	None	[{ ip: string, mac: string, hostname: string, vendor: string }]	Scan the network via router to find connected devices

Table: /api/wifi

Relative URL	Method	Required Body	Data section	Description
/enable	POST	None	message: string	Enable WiFi on the router
/password	POST	{"password": "<new_password>"}	message: string	Change the WiFi password for the wireless interface
/status	GET	None	enabled: bool, ssid: string, encryption: string	Get the current WiFi status including enabled state and SSID
/ssid	GET	None	ssid: string	Get the current WiFi SSID
/ssid	POST	{"ssid": "<new_ssid>"}	message: string	Change the WiFi SSID (network name)
/disable	POST	None	error: string	Disable WiFi on the router (not currently implemented)

Table: /api/monitor

Relative URL	Method	Required Body	Data section	Description
/current	GET	None	[{ mac: string, ip: string, download: number, upload: number, unit: "MB", connections: number }]	Get usage information about all devices for today
/last-week	GET	None	[{ mac: string, ip: string, download: number, upload: number, unit: "MB", connections: number }]	Get usage information about all devices for the last week
/last-month	GET	None	[{ mac: string, ip: string, download: number, upload: number, unit: "MB", connections: number }]	Get usage information about all devices for the last month
/device/(query: period=current week month)	GET	None	{ mac: string, ip: string, download: number, upload: number, unit: "MB", connections: number, period: string, timestamp: string }	Get usage information about a specific device by MAC

Table: /api/agh

Relative URL	Method	Required Body	Data section	Description
/categories	GET	None	{ categories: string[] }	List available AGH categories
/categories	POST	{ category: string, domains: string[] }	{ category: string, bytes: number }	Create a new category with initial domains
/categories/<category>/domains	GET	None	{ category: string, domains: string[], count: number }	Get domains for a category
/categories/<category>/domains	PUT	{ domains: string[] }	{ category: string, bytes: number }	Replace full domain list for a category
/device/rules?mac=...&ip=...	GET	None	{ categories: string[], client: { name: string, ids: string[] } }	Get effective categories for a single device
/devices/rules	POST	{ devices: [{ mac?: string, ip?: string }...] }	{ results: [{ input: object, categories: string[], client: object }] }	Bulk get effective categories for devices
/device/rules	POST	{ device: { mac?: string, ip?: string }, categories: string[] }	{ updated: number, details: object[], apply: object }	Set categories for a single device
/devices/rules/set	POST	{ devices: [{ mac?: string, ip?: string }...], categories: string[] }	{ updated: number, details: object[], apply: object }	Set categories for multiple devices
/device/rules	DELETE	{ device: { mac?: string, ip?: string } }	{ cleared: number, apply: object }	Clear all rules for a single device
/devices/rules	DELETE	{ devices: [{ mac?: string, ip?: string }...] }	{ cleared: number, details: object[], apply: object }	Clear all rules for multiple devices

Table: /api/bandwidth

Relative URL	Method	Required Body	Data section	Description
/limits/group	POST	{ ips: string[], download_kbytes?: number, upload_kbytes?: number, download_mbps?: number, upload_mbps?: number }	{ applied: number, deleted: number }	Apply bandwidth limits to a group of IPs (Mbps auto-converted)
/limits/group	DELETE	{ ips: string[] }	{ deleted: number }	Remove limits for a group
/limits/device	POST	{ ip: string, download_kbytes?: number, upload_kbytes?: number, download_mbps?: number, upload_mbps?: number }	{ ip: string, dl_kbytes: number, ul_kbytes: number, deleted: number }	Apply per-device limits
/limits/device	DELETE	{ ip: string }	{ deleted: number, ip: string }	Remove per-device limits

MIDDLEWARE NOTES:

- All endpoints except /health require sessionId and routerId parameters (via query params or JSON body)
- All endpoints except /health and /api/session/start require an active session

Response format:

All responses are wrapped in a standard envelope:

```
| { "success": boolean, "data": any, "error": { code: string, message: string, details: any } | null, "metadata": { sessionId: string | null, routerId: string | null, timestamp: string, executionTime: number } } | | :---- |
```

- The before_request hook (verify_session_and_router) runs before all requests to validate session context
- Session context is stored in Flask's g object (`g.session_id`, `g.router_id`) for access throughout request lifecycle